

Formally Modeling CI/CD

Ava Crocker
acrocker2021@my.fit.edu
Florida Institute of Technology
Melbourne, Florida, USA

Taylor Carlson
tcarlson2021@my.fit.edu
Florida Institute of Technology
Melbourne, Florida, USA

Tommy Gingerelli
tgingergelli2021@my.fit.edu
Florida Institute of Technology
Melbourne, Florida, USA

Daniel Couturier
dcouturier2021@my.fit.edu
Florida Institute of Technology
Melbourne, Florida, USA

Abstract- Continuous Integration and Continuous Deployment (CI/CD) are flows/pipelines that software systems use to build, test, and deploy their products. This allows for seamless integration of changes without the intervention of human error. Although CI/CD pipelines are effective, they often go unutilized due to manual methods and a lack of trust in the automation. This paper aims to expand on the preliminary idea of formalizing CI/CD systems, research ten other studies within this topic, and note any successes or shortcomings.

1 Introduction

1.1 Background

In modern software development, developers often work simultaneously on separate components, which may be further divided into several smaller pieces called branches. All branches are expected to integrate successfully into the final product, but branches may receive several updates over time as developers finish work or requirements change. As projects grow in scale and complexity, the likelihood of overlapping changes, incompatible implementations, and conflicting ideas rises dramatically, introducing the potential for bugs, regressions, and security vulnerabilities. As a result, testing complex systems becomes increasingly important, but also increasingly difficult. In such cases, manually coordinating and merging contributions from multiple developers can be error-prone, time-consuming, and ultimately unsustainable.

To address these challenges, many organizations employ Continuous Integration and Continuous Deployment (CI/CD) pipelines - processes designed to streamline and standardize the processes of building, testing, and deploying code. CI/CD systems allow developers to commit changes frequently, with automated tools validating those changes through

regression testing, static analysis, and runtime checks before they are integrated into the main codebase. By automating validation and deployment, CI/CD pipelines aim to accelerate development, reduce human error, and ensure that the software remains in a releasable state at all times. This works well regardless of project size, which is especially valuable for large teams.

1.2 Motivation and Goal

While CI/CD pipelines offer significant benefits to the development lifecycle, they are themselves complex software systems. Like any other software, they can suffer from design flaws, implementation bugs, configuration errors, or security vulnerabilities. A failure in the CI/CD system might allow code that fails to test or build to be integrated into the main codebase, breaking a critical feature or causing a general system outage. As a result, developers and organizations may hesitate to fully trust the automation provided by these pipelines, opting instead to manually review changes, which hinders development efficiency, which begs the question:

How can we validate intermediate changes to a repository to ensure unstable code does not violate safety properties of the application in production?

This project aims to answer this question by applying formal methods to validate the automated test, build, and deployment processes. Formal methods involve using mathematically rigorous techniques to validate software systems. By creating formal specifications of the system, one can analyze the behavior under all possible scenarios and prove correctness properties such as consistency, safety, and liveness. Specifically, this project will validate that under reasonable assumptions, branches must be tested, pass tests, and build successfully before they can be integrated safely. Further, the integration process must never cease, or cause the system to cease, and must give all branches an opportunity to commit. By ensuring

these properties of the system hold, it reduces the risk of integration errors and contributes to the assurance of the CI/CD process that cannot be met with traditional testing or manual review. As a result, developers and project managers should feel more confident in adopting CI/CD systems in new or existing projects, contributing to the safety and security of the software industry as a whole.

2 Related Work

Properly modeling continuous integration/continuous deployment systems is crucial for software developers. Having a formalized product eliminates ambiguity and ensures seamless development. Many other researchers have noticed a similar need due to the lack of formalized CI/CD systems. The study aims to discuss the findings of ten researchers and papers and analyze how their work can impact future studies.

CI/CD lacks formal modeling because there aren't studies that model the overall benefits of utilizing a CI/CD pipeline. Researchers developed a Petri net model to formally depict and analyze a CI/CD pipeline to improve the odds of successful delivery in the paper "A Stochastic Petri net Model of Continuous Integration and Continuous Delivery" [1]. The stochastic Petri net model has performance and sensitivity analysis performed after development to prove statistical accomplishments within its models. The models proved to improve successful product delivery. While the paper doesn't discuss specific limitations, it notes that the implementation in their paper aims to mitigate previous limitations done by other studies, such as analyzing implementation/modification [1].

Researchers developed a tool called Rig to support creating workflows to model specific instances, such as a CI/CD pipeline. The study "Evaluation of Graphical Modeling of CI/CD Workflows with Rig" models the CI/CD pipeline in their self-developed tool to demonstrate the basic concept of the pipeline and reveal the challenges of the current implementation [2]. The research presented in this study lacked a control group; therefore, there was no data to compare the findings. This can be a limitation because the statistical results may not be accurate [2].

DevOps and CI/CD paradigms need to be efficiently implemented with formal methods, and if they cannot, they don't substantially impact the software development process. Researchers in the study "Enabling continuous integration in a formal methods setting" [3] aim to investigate this statement. The Vienna Development Method using the Overture tool

is applied to their research. VDM models were created and then analyzed by formal proof, aiming to meet specific parameters: coverage, speed, and system integration. The one main limitation of this research is that it becomes challenging to analyze large models [3].

Many critical errors are encountered when using CI/CD for software development. Researchers in the paper "A Model-driven Approach to Continuous Practices for Modern Cloud-based Web Applications" aim to model this process to generate guaranteed success formally [4]. Meta-modeling framework CINCO was used for their model-driven approach to graphically depict CI/CD systems. The model was applied to cloud-based CI/CD's, allowing developers to model specific tasks and dependencies. A limitation of this approach was found to be that for proper model-checking, it needs to be unfolded to get the transition system. This limitation didn't hinder any testing results [4].

Continuous Integration is present in almost every large and small-scale industry organization. The paper "Industry Application of Continuous Integration Modeling: A Multiple-Case Study" dives into that fact and uses concrete examples of four industry cases in three companies to model their continuous integration techniques [5]. This study used and analyzed two modeling techniques, Continuous Integration Visualization Technique (CIViT) and Automated Software Integration Flow (ASIF). Two main limitations were recognized: identification/planning, and lack of support with technical work [5].

One factor that a researcher found in "Modelling CI/CD Pipeline Through Agent-Based Simulation" that helps build CI/CD pipeline models is an agent-based simulation [6]. The models tested in real-world environments are expensive and complex, so they find a way around it by using the simulation to be cost-effective. They also found that specific technologies used in the model can cause issues; however, with the model they used in the simulation, the model is in a neutral environment. The researcher saw limitations with modeling CI/CD pipelines, so they believed in using an agent-based simulation to help create one to lower the limitations [6].

Continuous integration is very detailed, which makes it difficult to generalize the structure, so detailed documentation is needed to create the model effectively. "Modeling continuous integration practice differences in industry software development" talks about the limitations of the

different ways to develop the model. Still, it believes that stating the main points of the structure will help create a simple but effective way to make the model. The model contains the integration flow anatomy and the node attributes [7].

One researcher in “Impediments to Introducing Continuous Integration for Model-Based Development in Industry” believed that modeling continuous integration was a practice that would help improve development productivity [8]. One company in the research study believed that implementation would make continuous integration modeling difficult. The company thought that the problems would prohibit any increased productivity. The paper lists different impediments identified in companies through modeling continuous integration, including a lack of modeling experience among workers and the need for model validation. The barriers identified in modeling continuous integration include functional, non-functional, human, and business aspects. Some future work comprises companies that use continuous integration differently [8].

Modeling solutions for CI/CD involves many different difficulties. First, “Dual-Modeling Approaches in CI/CD” talks about the lack of standardization and how different pipeline tools lead to inconsistencies in modeling techniques [9]. Second, the issue is finding workers who can create a model and have knowledge of both development and operational aspects. Third, high-level models do not mean better software will exist, and can cause complex situations. Fourth, the model evolution can cause errors if any updates are not changed within the model and do not meet the requirements [9].

One researcher in “Continuous Integration in Multi-view Modeling: A Model Transformation Pipeline Architecture for Production Systems Engineering” believed that model-driven engineering has to incorporate code changes into the model and the system development. To maintain accuracy, unit tests need to check the model, and continuous synchronization will help make the model efficient and help with changes [10].

3 CI/CD Modeling Analysis

The plan is to independently model CI/CD GitHub components, specifically GitHub Actions and Workflows. These open-source tools are a widely accessible and free set of tools that can be used for CI/CD. The immediate impression of GitHub is that it is only a version control tool. It has many sub-components that allow for CI/CD integration.

GitHub Actions provides developers with a CI/CD tool that incorporates every process step, from build to test to deployment. Having this tool within Git's capabilities is extremely useful for developers, primarily when GitHub acts as the version control tool. Git Workflows is another crucial tool within Git for developers. Workflows consist of automated tasks kicked off by a specific event, such as a push to a repository. Actions and Workflows work effectively to incorporate a seamless CI/CD process.

The plan is to validate each tool independently and their exchange of information to verify the integrity from source code commit, through building, testing, and finally, deployment of versions. Therefore, accurately modeling the CI/CD logic flow is crucial. A model project using formal methods will be designed with a set of test cases to pass before being published to a GitHub Pages site. The model in Section 4.1 accurately depicts the flow of actions. The goal is to have the flow of CI/CD completed before publishing the accomplishments to Git Pages. These changes will be examined to verify that each process executed correctly as the model(s) expect. Then, the results of this test will be cross-validated with the expected results, and any edge cases or catch states will be found.

3.1 Diagram

The figure below [Figure 1] demonstrates the flow of processes expected to be formally modeled. While focusing on the CI/CD components, it's essential to integrate the factors that happen before and after. Considering this, the figure models the flow starting at Git, moving to Git Build Actions, then to Git Testing Workflows, then finally publishing to Git Pages. Within these boxes lies more of the entire process. The Git box represents the user's system. Here is the Main repository and the user's local branch. The Git Action box contains the system waiting for the user's actions to initiate the workflow. This includes waiting for a commit and push to the shared repository. Once Git Actions receives a push, the system moves to the workflow state. The Git Workflow box will initiate the build, test, and deploy section. While having the test branch off into its unit test sub-section, handling pass and fail cases differently. Once the workflow has reached the end of its process, it will successfully release the work to Git Pages, where all of the information is stored.



Figure 1

4 Formal Modeling

UPPAAL was used to model and formally simulate action flow within a CI/CD system. UPPAAL is an integrated tool that allows for visual models of the system's model, validation, and verification [11]. With this tool, the contents and specifications of Git Tools, including branches, repositories, Git Actions, Git Workflow, and Git Pages, were represented.

To accurately accomplish and depict the proper flow of a CI/CD system, two branches were simulated pushing code to a repository. Each process would continue individually, ensuring two commits and workflow actions wouldn't coincide and override each other. The pipeline flow will be initiated once the commit and push are received. The pipeline would then execute and test, check for test pass or failure, move to build, and then to deploy. If the test and build failed continuously, the system would return to production and stay in that state. To ensure no two commits enter the workflow simultaneously, there is a controller in the system to monitor and wait for process completion.

Each process present within the UPPAAL simulation was crucial for the CI/CD system. The individual processes include updates, guards, synchronization, and parameter intake. To represent the existence of passes and failures within development, since the outcome of a test is not random, there are parameters for each branch to determine if it will pass or fail.

The system heavily relied on channels to signal each process to wait or execute. This realistically represents the true nature of a CI/CD system because the flow of actions is not automatic. It requires the system to wait and receive proper notification that the valid information is present, and the system can move forward with the specified task.

The following sections describe each process: Controller, Branch, Repository, CI/CD Pipeline, Unit Test, and Page. From there, the system declarations will be elaborated on to explain how the system works together.

4.1 Controller

The Controller process handles two different branches to ensure they do not run simultaneously. It is critical to maintain process integrity during the Git process, where states cannot be accessed at the same time. As seen in Figure 2, the system's initial node is Idle, and the process waits until the branch A is

triggered and sends a synchronization signal on `runBranchA` to indicate the start of the Branch A execution.

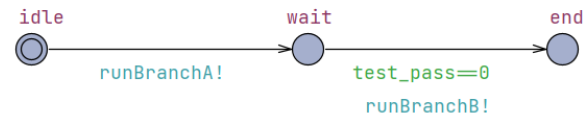


Figure 2

The process then goes to the Wait location, where it stays until the branch B location is triggered and starts. There is also a guard that checks to make sure that `test_pass` is equal to 0. The `test_pass` variable is set to 0 when a branch is deployed and sent back into production. The edge helps with checking that Branch A is deployed and complete before Branch B can start the process. When Branch B is verified, then the location will be moved to End, where it is then complete.

The main goal of the Controller process is to prevent conflicts between branches and maintain the transitions correctly. Two Git branches cannot be deployed simultaneously, so the Controller process is needed to ensure reliability and integrity within the system.

4.2 Branch

The Branch process is to portray the branch being committed to production through deployment for the Git process. As seen in Figure 3, the system's initial node is Idle, and the edge includes the `commitCode` being set to 1 and the `test_pass` is set to 1 in Branch A to signal that Branch A passes. In the CI/CD pipeline, if the branch passes, then the branch will be deployed and sent into production. However, Branch B is a branch that purposefully fails, and the `test_pass` is set to -1. In the CI/CD pipeline, if the branch fails, then the branch will be sent back to production to start the process over again. The synchronization signal, `run?`, is used in each branch to be used in the Controller process to ensure that the branches are not run simultaneously.

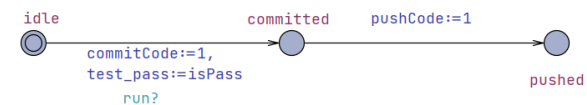


Figure 3

After the edge, the process is in the location Committed, where it waits and then sets `push_code` equal to 1. When `push_code` is equal to 1, then the Repo process sends the code to be validated. After the edge, the process is then moved to the location Pushed. The locations provide checkpoints needed to complete the key stages in the Git process, allowing the system to simulate a branch moving from commit to deployment.

The two different branches are to simulate and test the pass and fail of a commit during the Git process in our UPPAAL model. Branch A is to test what will happen to a branch when the commit passes testing, and Branch B is to test what will happen when the commit fails testing. The process ensures that deep testing is complete and multiple different scenarios are run in the model.

4.3 Repository

The Repository process handles receiving a commit and push from a branch and sending the information to the CI/CD pipeline.

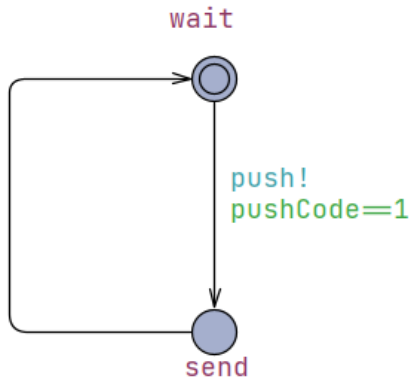


Figure 4

As seen in Figure 4, the system's initial node is at Wait. The process sits at Wait until the `pushCode` variable is one. Once the variable has the expected value, the edge triggers a synchronization signal on the Push channel.

The leading edge in this process also acts as a security barrier within the system. It's known that within the Git process, the user has to validate their credentials before any data gets transferred. This ensures all changes sent to the repository are authorized and validates that malicious code isn't sent to a shared location. While no specific node or edge waits for the user's login, the guard and channel act as that security barrier. Having the guard check that `pushCode` is one acts as a verification step, checking that the push met the preconditions within

authentication. If there is no authentication and the variable doesn't meet the expected result, the code doesn't get sent to the main CI/CD workflow.

Once the process reaches the Send node, the previous edge has completed all required actions, and the system continues as expected. The Send node then loops back to the Wait node on a blank edge, and sits at the Wait node until it receives another push.

This process represents the intermediary steps between the user committing and sending code to a repository, and the repository sending signals to the workflow that information has been received.

Concerning the initial system diagram in Figure 1, the Repository process represents both the Git block and the Git Actions block. The repository itself is encapsulated in the Git block. The edge actions of waiting for authentication and confirmation that the push was sent, and the Push channel sending a synchronization signal to the CI/CD pipeline represent the Git Actions block. This is because, as previously mentioned, the Git Actions block simply acts as a process waiting for information. Once that information is received, it sends a signal to initiate the workflow.

4.4 CI/CD Pipeline

The CI/CD Pipeline process handles the most critical actions of the system as a whole. This process validates the code sent to the workflow by testing, building, and deploying it. It also includes a node for code production. This is where the simulation continuously loops after passing or failing code is sent to the workflow. This represents the user continuing development before the next push.

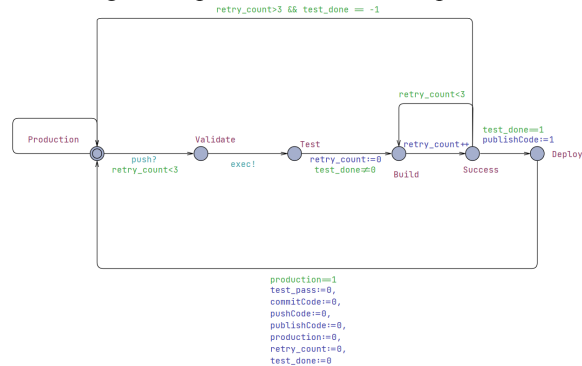


Figure 5

As seen above in Figure 5, the CI/CD Pipeline process contains a mainly linear flow with additional loops handling the pass and fail cases.

To begin at the initial Production node, as previously stated, this node represents the user in the development process. It'll reach and loop at this node at the end of the pass and fail processes to depict the user's further development.

The edge between the Production and Validation node includes two checks: one waiting for the synchronization signal on the Push channel, and one validating that the system hasn't exceeded three retries of the build. This ensures that the build process doesn't execute unless there is substantial code for it to run through the CI/CD pipeline. If there's not substantial content, it will loop through production or continue the system elsewhere at an available node.

From Validation to Test is an edge that initiates the Exec channel. This channel sends a signal to the unit test process while waiting to begin its test. The tests can only start once they are notified that execution is allowed and the received code has been validated. Further information about the Unit Test process is detailed in Section 5.5.

Once the system tests are complete, it moves to the build phase. To transition to building, the edge conditions have to be met. Between the Test and Build nodes are an update and a guard. The update sets the value of `retry_count` to zero. This ensures that the count starts from the beginning as it enters the building loop. The guard on the edge checks that the `test_done` variable is not equal to zero. This check relates to the description in Section 5.5, but to briefly add context, after the unit tests are complete, the process sets variable values to `test_done`. The guard wants to ensure that the tests are complete before initialization of the build.

The transition between the Build and Success node is non-deterministic if the Unit Test process passes. It can either increment `retry_count` and move on, or loop back to build and try to rebuild. This is not a bug in the system; it simply represents cases where the test passes but the build may need to try again due to external circumstances impacting the output, i.e., a computer outage. If it doesn't move on and loops, the loop edge has a guard checking that it hasn't retried the build more than three times.

Once the rebuilds on a passing Unit Test process reach three, the Success node will move to Deployment. This transition happens based on the edge checks of verifying that `test_done` is one, i.e., that the unit test is completed and passed, and it also

updates the `publishCode` variable to one, allowing the system to move to a new process.

After reaching the Deploy node, the system will move to the Page process detailed in Section 5.6. The edge from Deploy to Production has a main check to verify that publishing is done and the system can return to production. In addition to this check, it also resets all system variables back to zero. This allows more branches to enter all processes and continue with the CI/CD system.

If the Unit Test fails, the system will still move on and try to build, but because the code isn't ready for deployment, the build will fail. Due to the build failure, the system will again try to rebuild the code at least three times. Once the rebuilds on a failed Unit Test process exceed the specified threshold, the system will be looped back to production for further code development.

In relation to the system flow in Figure 1, this process acts as the Git Workflows block. As earlier described, the workflow handles testing, building, and deploying. This block is the main component of the CI/CD Pipeline.

4.5 Unit Test

The Unit Test process is to ensure that the branch commits can both fail and pass, and not cause the model to break or go into deadlock. In Figure 6, the initial state of the system is Start, and checks for a synchronization signal when the CI/CD pipeline moves from the Validate stage and the Test stage.

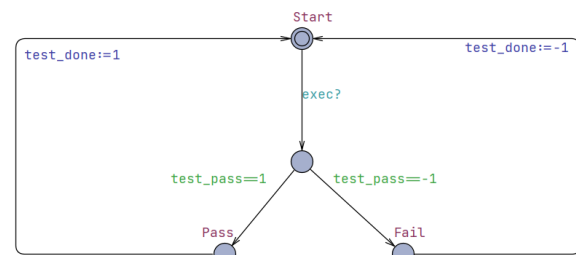


Figure 6

Then, the process checks if the `test_pass` variable is equal to 1 or -1. Branch A has the `test_pass` variable equal to 1, so it then has `test_done` equal to 1. In the CI/CD pipeline, if the `test_done` variable is not set to 0, then the stage moves to Build. However, when `test_done`

is equal to 1, then it will move to the success stage, but if test_done is equal to -1, then it goes back into production to start the process over again because it failed.

The Unit Test process is important to test to ensure that both pass and fail tests. The UPPAAL model needs to be tested with both fail and pass test cases to ensure reliability.

4.6 Page

The Page process concludes the system by publishing all code changes. These changes represent what was sent to the repository and what initiated the workflow. At the conclusion of the CI/CD Pipeline, the last step is to load the changes to the program website, which is what the Page process does.

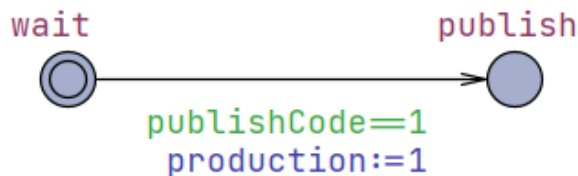


Figure 7

The Page process is a very small system containing only two nodes, as seen in Figure 7. The first node, Wait, has an edge that goes to the Publish node. The edge between these two nodes has a guard and an update.

In order for the Wait node to be initiated and transition to publish, the publishCode variable has to have a value of one. This value gets set in the CI/CD Pipeline process described in Section 5.4. The publishCode variable is initially set to zero and only gets updated if the Unit Test process in Section 5.5 passes, and the build has moved to Success. Once the system moves from Success to Deploy, the publishCode variable is set. This represents the system verifying all outputted changes and confirming that everything was successful and can be published.

Once the edge receives verification that the code can be published to the website, the guard is enacted, and the system transitions to the Publish node. While transitioning to the Publish node, the production variable gets set to one. This notifies the system that the CI/CD process is complete, and the user can go back to production before the next code change and push.

This process represents the Git Pages block in the system diagram in Figure 1. As previously described, once the Workflow block successfully completes a passing loop, everything will be published to Git Pages.

4.7 System Declarations

To allow the system to work together as a whole, the global and system declarations are crucial to the output. All variables within the system are put into the global section because all of them need to be accessed by more than one process. As seen in Figure 8, all global variables are initially set to zero.

Regarding the two isPass values of 1 and -1 in BranchA and BranchB, these are the parameter values that get taken in when declaring the system. 1 in this case means the code from BranchA will pass, and -1 means the code from BranchB will fail.

To accurately simulate both outcomes of the entire CI/CD process, two Branch processes are initialized within the simulation.

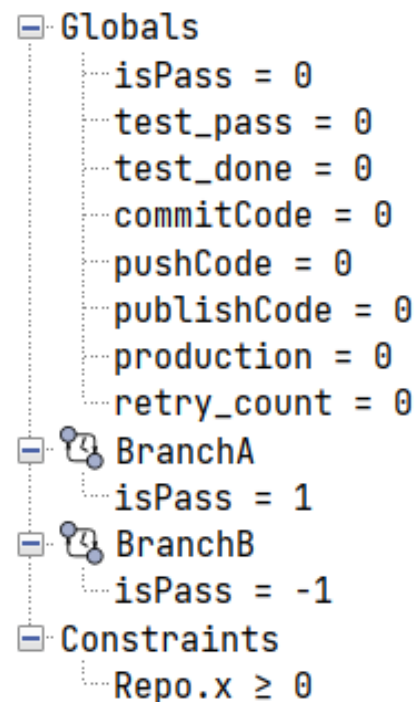


Figure 8

The flow of actions within the simulation exactly represents the realistic process of a CI/CD flow. Figure 9 shows the first four initial processes. Two branches: BranchA and BranchB, one repository, and one controller.

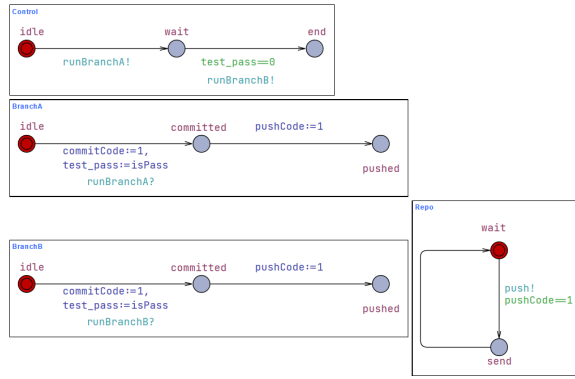


Figure 9

Each process is waiting at the initial red node and sits at the node until it receives a channel signal or guard satisfaction to continue. The Control process is the first to act, as it manages both branches to ensure they don't run through the CI/CD process at the same time. The Control will first initiate BranchA to run through the system, and then BranchA will send information to the repository.

The next step is for the repository to send its information to the Pipe process seen in Figure 10.

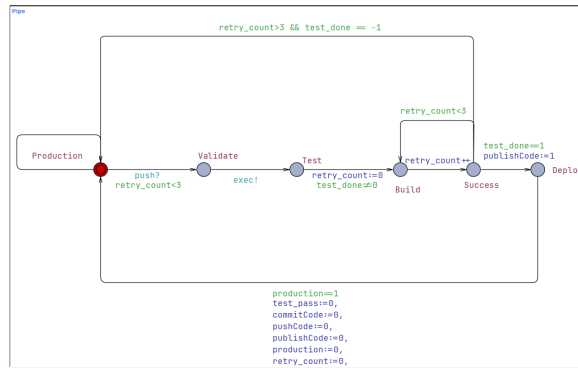


Figure 10

The Pipe process waits at the Production node until it receives the channel signal from Push and the guard condition is satisfied. From there, it will move through the pipeline as expected.

Once the system hits the Test node, it will then move to the Unit Test process. As seen in Figure 11, This Test process runs tests on both the failing and passing nodes, and then sends the information back to the pipeline.

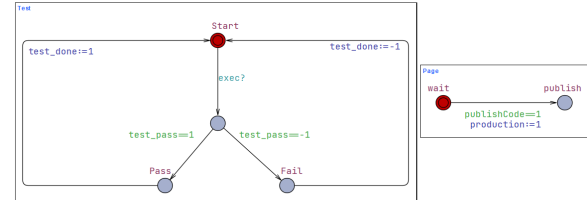


Figure 11

Once the Test process is completed and it returns to the Pipeline, the Page process waits for Deployment to verify and validate the system for publication.

After the code changes are published, the system returns to production in the Pipeline process. This represents the developer/user working on future code changes to add to the CI/CD system.

5 Processes Verified

To verify the integrity of the UPPAAL model, the system was analyzed and examined for adherence to formal properties. The ability for CI/CD to adhere to formal properties will allow us to finalize our conclusion that CI/CD is a safe and reliable system of deployment for software development.

Safety is a type of property that guarantees that a system does not execute a bad action. It is a prohibited state or response from a system that could cause cascading effects or system crashes. Safety properties in mission-critical systems that need to guarantee success, without the possibility of crashing or errors.

Invariance is a subset of the family of safety properties. Invariance is a special safety property that is used to ensure compliance of properties across the entire system. The required use case in CI/CD is present due to CI/CD's concurrent nature. Invariance in concurrent systems, such as CI/CD, ensures system integrity among multiple concurrent operations.

Liveness is a family of properties that guarantees eventual system behaviour. In an infinite trace, an expected execution trace will emerge. Liveness is critical in guaranteeing the successful operations of a system like CI/CD, which needs to eventually update repositories and test commits.c

5.1 English-Based Representation

The first step of the CI/CD pipeline is for a developer to push code changes to the server; this action will kick off the CI/CD pipeline and therefore initialize the rest of the system. However, the rest of the

pipeline, such as the Test and Build phases, require code to verify. Therefore, the first safety property we will analyze is that *the system will not start Validation without a push*. In the UPPAAL model, Fig. 10, it is shown that the system cannot enter Validate until Repo authorizes it using push.

Once we enter the validation phase, we need to guarantee that another branch will not begin the push process until after our code has concluded the steps to production. If new code is pushed during the process of testing or building the application, then there is a potential that new unvalidated changes could break our software system in production. To solve this issue, we assert that *new code will never be pushed before the current changes have moved to production*. This safety property, Fig. 9, demonstrates the inability of CI/CD to publish code that has not gone through testing and the building process of our pipeline.

One such concern around CI/CD is the ability for it to accurately handle multiple different pushes/commits asynchronously. In the previous paragraph, we guaranteed that no new code would interfere with existing code in the pipeline. Now we must handle the new code that has been committed on BranchB. To guarantee fairness, our system guarantees *BranchB will eventually become live*. This liveness guarantee can be seen in the Control model, Fig. 9, where control is responsible for queuing branches with committed code to the pipeline, ensuring that every branch in the queue gets a turn.

Another primary concern of automated systems' ability to move code to production is the ability to handle the build process of an application. System builds are a very integral process for the deployment of an application with many moving parts that could fail. The designed model takes into account issues in the build process and commits to the invariance property, *every build will always get at least 3 build attempts*. Three builds are a sufficiently large number of attempts that if a build were to fail, it is due to a design or development issue in the new code rather than the model's behavior.

With the guarantee that every code will eventually be piped through the pipeline, we must now address the issue about the system's ability to return to production. Fortunately, in the design of CI/CD, there is no set of traces that does not result in the return to production. The liveness property of *all built code will eventually return to production* in conjunction with the invariance property; *code that enters validation will always be built*, guaranteeing that any

code that enters the CI/CD pipeline will always be tested, built, and returned to production.

5.2 Formal Representation

As safety and reliability is of the utmost importance in software systems, it is equally important in the methods we use to deploy software. Seen below, Fig. 11, is the formal representation of the safety property *no code that fails testing will move to deployment*, found in the pipeline of CI/CD.

```
A[] not (test_pass = -1 and Pipe.Deploy)
```

Figure 11.

Continuing on the topic of safety, it would be extremely dangerous if a critical bug fix couldn't be released due to the pipeline getting stuck building. Figure 12 demonstrates the formalization of the safety property; *a build will never be infinitely stuck in the build process*. The figure demonstrates the incorporation of a retry counter. This retry counter pertains to the number of builds done, and it cannot exceed more than 3 builds.

```
A[] retry_count ≤ 3
```

Figure 12.

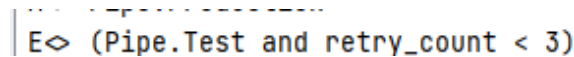
When adding changes to a codebase on a professional level, it is necessary to validate those changes before moving them to deployment. In this manner CI/CD is there to assist a company in verifying modifications to a codebase. In this effort, the invariance property *production will eventually move validation*. As seen in Figure 13, any changes on a branch will cause production to eventually move to validation. The figure represents that if the build retry doesn't exceed its limit, there will be a case where the code enters the pipeline and validates.

```
E◇ (Pipe.Validate and retry_count < 3)
```

Figure 13.

Once in validation, the actual process of validating code is a multi-step process. The first

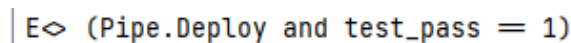
part of validation that new changes must undergo is testing. Testing is a core process in the continuous integration / continuous development pipeline. This means that verifying the process of moving from validation to testing is also a critical step in the CI/CD formal model. As seen in the figure, figure 14, our model holds that *eventually code in validation will move to testing*. The figure represents that if the build retry doesn't exceed its limit, there will be a case where the code enters the pipeline and tests.



```
E > (Pipe.Test and retry_count < 3)
```

Figure 14.

Finally, the last step of CI/CD that needs to be formalized to complete our modeled implementation is the ability to reach deployment. Deployment is the final part of the pipeline and where code goes from an artifact of a state in the model. To become a part of the model itself. As seen in Fig. 15, the formalization of deployment shows that *the deployment process is eventually reached from a passing build to entering deployment*, where the code becomes a part of production and the pipeline will be reset for the next commit.



```
E > (Pipe.Deploy and test_pass = 1)
```

Figure 15.

6 Conclusion

As software systems become increasingly complex, CI/CD pipelines have proven to be a necessity in software engineering, offering automation and speed in managing code integration and deployment. However, their high complexity introduces new risks that can sometimes undermine the very stability they are meant to ensure. This project has successfully demonstrated the applicability of formal methods as a systematic approach to validating the correctness of CI/CD processes.

In our initial model, we created a basic model of the CI/CD pipeline with actors including the main system, the main branch, new branches produced by programmers, and the testing suite for each branch. With this model, we were able to demonstrate several basic safety and liveness properties. The first of these was that the program must not exist in a state where it is trying to build the software, but the incoming

branch has failed a test case, which would result in unstable code reaching production. Second, we validated that a successful branch would eventually be deployed, ensuring the system doesn't hang in any state. Finally, we determined that all branches would be able to eventually succeed if they had ideal execution in the pipeline, showing the pipeline would accurately evaluate submissions.

In the final model, we added the ability for multiple retries in each state, which is represented by the resilience of the system and programmers when errors occur. In this new model, we were able to demonstrate the above properties and several new ones. In addition to ideal states, we were able to show that the system does not deadlock under any circumstances, which would prevent the ability for new changes to be integrated. We also demonstrated the safety of the pipeline to prevent two conflicting branches from accessing critical sections of the pipeline at the same time, which could cause issues. Finally, we showed that retrying in some states does not continue forever, which ensures some level of fairness between branches, and liveness to get to all branches in a reasonable time frame.

Overall, the results of this work suggest that formal verification can complement manual testing and review strategies, enabling greater confidence in automated development tools. In doing so, it empowers software teams to embrace CI/CD practices more fully, ultimately supporting faster, safer, and more scalable software delivery.

7 Citations

- [1] S. Bhadra, "A Stochastic Petri net Model of Continuous Integration and Continuous Delivery," 2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), Charlotte, NC, USA, 2022, pp. 114-117, doi: 10.1109/ISSREW55968.2022.00050. keywords: {Analytical models;Sensitivity analysis;Pipelines;Petri nets;Stochastic processes;Software;Product design;CI/CD pipeline;petri net;stochastic petri net;performance;sensitivity analysis}
- [2] Teumert, S. et al. (2022) 'Evaluation of graphical modeling of CI/CD workflows with rig', Lecture Notes in Computer Science, pp. 374–388. doi:10.1007/978-3-031-19756-7_21.
- [3] Luis Diogo Couto, Peter W. V. Tran-Jørgensen, René S. Nilsson, and Peter Gorm Larsen. 2020. Enabling continuous integration in a formal methods setting. *Int. J. Softw. Tools Technol. Transf.* 22, 6 (Dec 2020), 667–683. <https://doi.org/10.1007/s10009-019-00546-y>
- [4] T. Tegeler, F. Gossen and B. Steffen, "A Model-driven Approach to Continuous Practices for Modern Cloud-based Web Applications," 2019 9th International Conference on Cloud Computing, Data Science & Engineering (Confluence), Noida, India, 2019, pp. 1-6, doi: 10.1109/CONFLUENCE.2019.8776962. keywords: {Pipelines;Syntactics;Servers;Cloud computing;Software;Task analysis;Tools;Continuous Integration;Continuous Deployment;Model-driven;Cloud-based Applications}
- [5] D. Ståhl and J. Bosch, "Industry Application of Continuous Integration Modeling: A Multiple-Case Study," 2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C), Austin, TX, USA, 2016, pp. 270-279. keywords: {Software;Industries;Visualization;Companies;Analytical models;Context;Software engineering;Agile software development;software integration;continuous integration;automation;modeling;visualization}
- [6] Q. Liao, "Modelling CI/CD Pipeline Through Agent-Based Simulation," 2020 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), Coimbra, Portugal, 2020, pp. 155-156, doi: 10.1109/ISSREW51248.2020.00059. keywords: {Pipelines;Software;Automation;Testing;Biological system modeling;Analytical models;Tools;CI/CD;Agent-based Modelling;Queueing System;Pipeline Simulation;Multi-agent Simulation},
- [7] Daniel Ståhl, Jan Bosch, Modeling continuous integration practice differences in industry software development, *Journal of Systems and Software*, Volume 87, 2014, Pages 48-59, ISSN 0164-1212, <https://doi.org/10.1016/j.jss.2013.08.032>. (<https://www.sciencedirect.com/science/article/pii/S0164121213002276>)
- [8] R. Jongeling, J. Carlson and A. Cicchetti, "Impediments to Introducing Continuous Integration for Model-Based Development in Industry," 2019 45th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), Kallithea, Greece, 2019, pp. 434-441, doi: 10.1109/SEAA.2019.00071. keywords: {Companies;Interviews;System analysis and design;Industries;Productivity;Software systems;Model-based development, Continuous integration, Interview study}
- [9] A. R. Oliveira, Dual-Modeling Approaches in CI/CD: An Agnostic Modeling Framework to Streamline DevOps, BSc thesis, Computer Science and Engineering, NOVA University Lisbon
- [10] F. Rinker et al., "Continuous Integration in Multi-view Modeling: A Model Transformation Pipeline Architecture for Production Systems Engineering," in *Proceedings of the 2021 International Conference on Model-Driven Engineering and Software Development (MODELSWARD)*, 2021.
- [11] G. Behrmann, A. David, and K. G. Larsen, "UPPAAL — a Tool Suite for Automatic Verification of Real-Time Systems." [Online]. Available: <https://uppaal.org>.
- [12] Grammarly. Grammarly, Inc., <https://www.grammarly.com>.